

———— // ZPWR-MIDI-FX // ————

zpwr-midi-fx - Block Reference

v0.30

MODULE REFERENCE

note-stream MIDI effect modules

2026-06-23

MenkeTechnologies

———— MENKETECHNOLOGIES ————

Contents

Catalog at a glance	1
Architecture	1
Performance	1
Reading this catalog (badge key)	3
MIDI (zpwr-midi-fx) (111 blocks)	3
Index (111 blocks, alphabetical)	14

Catalog at a glance

111 blocks shipped by zpwr-midi-fx across 10 categories, including 0 component-level analog-circuit models and 45 control / modulation (CV-badge) sources. This is the per-plugin subset of the shared zpwr-patch-core catalog.

Registry MIDI 111

Output jack Audio 0 MIDI 111 CV 0

By category (block count, largest first):

Sequencing.....	39	Control.....	5
Harmony.....	21	Voicing.....	5
Modulation.....	13	Expression.....	3
Routing.....	11	Tuning.....	3
Dynamics.....	10	Probability.....	1

Architecture

Every block in this catalog runs through one shared patch-graph engine. The full signal and control path - from the host buffer, through the layered engine and the per-voice patch graph, onto the shared bus, and back out - is below. Signal flows top to bottom; control and modulation enter from the right; the engine's CPU optimizations are called out where they apply.

Performance

Every block in this catalog runs through one shared patch-graph engine, so every optimization in that engine speeds up every patch. The full list of CPU work shipped to date is below, grouped by what it attacks. The cost of one audio buffer is voices x samples x modules: the multiplier (voices x per-voice work) is what pins a chord; the slope (per-sample dispatch and coefficient recompute) is the constant overhead. The phases below attack each in turn. Measured figures are arm64, -O3, from interleaved before/after benchmark runs.

1. Engine hot path - shaves the per-sample slope; together ~18% off per-sample DSP:

- > Relaxed atomic ordering on every hot-path load (params, sync-divisor, gain, bypass, bus-send, modulation), dropping the arm64 acquire barrier on the inner loop.
- > Reused compute-context scratch - a single member buffer reused each sample instead of constructing one per sample.
- > Tempo-sync scan skipped for nodes that have no syncable parameter.
- > Per-block parameter snapshot - on the host's beginBlock call, unmodulated nodes do zero atomic parameter loads per sample; a safe per-sample fallback runs when a host never calls it.

2. Coefficient & transcendental caching - block-rate values that used to be recomputed every sample are now computed once per block and reused, recomputed only when an input parameter changes (all bit-identical):

- > CachedSVF caches the state-variable (TPT) filter coefficients, skipping a per-sample tan; one subclass fixes every SVF-based block with no call-site edits. ChoirPad 3-voice: 106 to 76 ns/sample (~30%).
- > Additive partial-amplitude cache - a dense additive voice was running up to 70 pow per sample (14 partials x 5 internal voices); recomputed only on a Partial / Rolloff / Odd-Even change. 1140 to 600 ns/sample (~48%).

ZPWR-PATCH-CORE — FULL SIGNAL & CONTROL ARCHITECTURE

one shared engine: host → plugin → layered engine → per-voice patch graph → shared bus → output

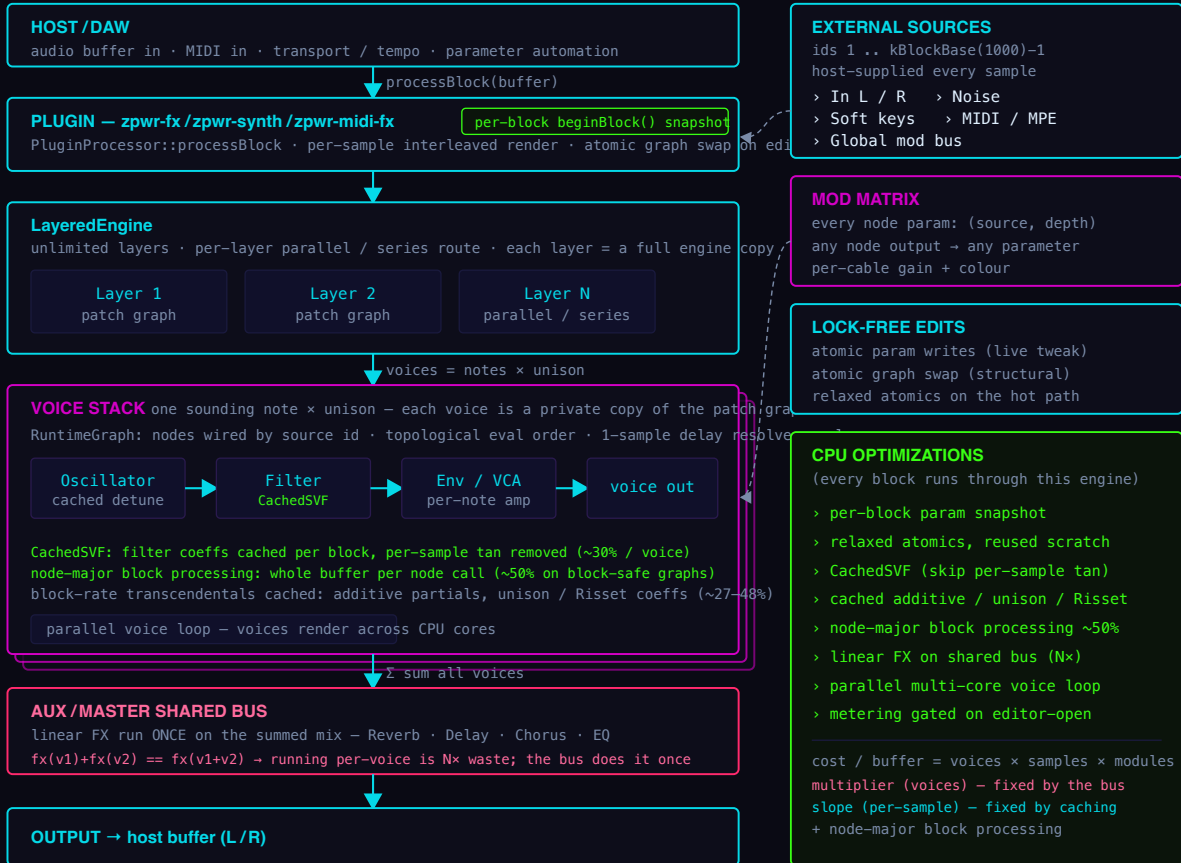


Figure 1: Full signal & control architecture of zpwr-patch-core

- > **Unison detune-ratio cache** - a **pow** plus a **sqr**t per voice per sample, cached once; benefits every unison oscillator. Osc 5-voice: 246 to 180 ns/sample (~27%).
- > **Per-voice detune-ratio cache** for Screech, Hoover, Reese, Wt, SynthFM and SynthSync (the per-voice **pow(2, spread x detune)** ratio).
- > **Risset decay coefficients** - the seven-partial **exp** decay loop cached.
- > **Additator amplitude-table cache** - up to 32 **pow** per sample on block-rate Partial / Tilt / Balance, cached like the additive table.

3. **Voice multiplier** - the real fix for chord cost; turns N copies of an effect back into one:

- > **Linear FX kept on the shared bus.** Reverb and delay are linear ($fx(v1)+fx(v2) == fx(v1+v2)$), so running them per voice is N-times waste for a bit-identical result. They live on the shared aux / master bus; only synthesis and per-note timbre stay in the voice patch.
- > **Every factory preset audited clean** - no linear FX sits in any voice layer.
- > **Regression gate** - the preset validator rejects any reverb placed in a voice layer, so the multiplier cannot creep back in.

4. **Host & parallelism** - removes wasted per-voice work in the synth host:- **Metering gated on editor-open** - the per-sample O(nodes) metering scan is skipped entirely while the editor is closed. - **Parallel voice loop** - voices render across CPU cores through a persistent worker pool, bit-identical to serial, opt-in.

5. **Block (node-major) processing** - each node processes a whole buffer in one call instead of one sample at a time, amortizing the per-sample cable-gather, context-build and dispatch overhead (~50% on block-safe graphs; a per-sample-capture fallback covers graphs with buses or modulation):

- > **11 FX blocks converted** to whole-buffer processing - the medium FX. Each mirrors its per-sample body over the buffer with block-rate coefficients hoisted out of the loop, and is verified bit-identical to the per-sample path. Converted so far: Bitcrush, Chorus, Comb, Drive, EQ, Filter, Flanger, Gate, Lofi, Tremolo, Waveshaper.

Reading this catalog (badge key)

An icon glyph precedes each block name; then its badges, then **in:** its input jacks and **out:** its output-jack type, then the description:

- > **[CAT]** category - **AMP** Amp, **ANA** Analog, **CTL** Control, **DLY** Delay, **DST** Distortion, **DYN** Dynamics, **ENV** Envelope, **FLT** Filter, **FX** Effect, **HRM** Harmony, **MOD** Modulation, **OSC** Oscillator, **PIT** Pitch, **PRB** Probability, **REV** Reverb, **RTE** Routing, **SEQ** Sequencing, **SPC** Spectral, **STR** Stereo, **TUN** Tuning, **UTL** Utility, **VOI** Voicing.
- > **[CV]** a control / modulation source (outputs CV, not audio).
- > **[CIRCUIT]** a component-level analog-circuit model (per-sample nodal / Newton solve).
- > **[FX]** **[SYNTH]** **[MIDI]** which **plugin(s)** ship the block - **zpwr-fx**, **zpwr-synth** (gets the whole audio pack) and/or **zpwr-midi-fx**.
- > **in:** the block's input jacks, and **out:** its output-jack signal type - **out: Audio**, **out: MIDI** (note stream) or **out: CV** (control / modulation).
- > **icon:** the glyph before each block name shows its kind at a glance:

	Oscillator		Filter		Delay
	Reverb		LFO / mod		Envelope
	Dynamics		Stereo		Pitch
	Spectral		Harmony		Sequencer
	Routing		Voicing		Effect
	Control		Probability		Tuning
	Utility		Guitar amp		Pedal / stompbox
	Speaker cabinet		Analog circuit		Other

MIDI (zpwr-midi-fx) (111 blocks)

Control (5)

⌚ **ATProc** [CTL][CV][MIDI] in: Audio out: MIDI

Scales aftertouch, optionally converting it to a CC.

⌚ **BendProc** [CTL][CV][MIDI] in: Audio out: MIDI

Scales and/or inverts incoming pitch-bend.

⌚ **CC2Note** [CTL][CV][MIDI] in: Audio out: MIDI

A watched CC crossing a threshold triggers a note.

⌚ **CCMap** [CTL][CV][MIDI] in: Audio out: MIDI

Remaps a CC number and scales/offsets its value.

⌚ **Note2CC** [CTL][CV][MIDI] in: Audio out: MIDI

Emits a CC from each note (pitch or velocity).

Dynamics (10)

⌚ **Accent** [DYN][MIDI] in: Audio out: MIDI

Boosts velocity on every Nth note.

⌚ **Humanize** [DYN][MIDI] in: Audio out: MIDI

Adds random timing and velocity jitter for a human feel.

⌚ **Ramp** [DYN][MIDI] in: Audio out: MIDI

Velocity ramp over a run of notes, ascending or descending.

⌚ **VelClip** [DYN][MIDI] in: Audio out: MIDI

Clamps velocity between Min and Max.

⌚ **VelCompress** [DYN][MIDI] in: Audio out: MIDI

Velocity compressor: velocities above Threshold are pulled toward it by Ratio (1 = off, 8 = hard), narrowing the dynamic range smoothly instead of hard-clamping like VelClip.

⌚ **VelCurve** [DYN][MIDI] in: Audio out: MIDI

Reshapes velocity through a power curve.

T **VelInvert** [DYN] [MIDI] in: Audio out: MIDI

Flips the velocity scale around its midpoint (soft <-> loud) by Amount, so your accents become ghost notes and the ghosts become accents - a velocity mirror, blendable to taste.

T **VelKey** [DYN] [MIDI] in: Audio out: MIDI

Keyboard velocity tracking: scales each note's velocity by how far its pitch sits from a Center note, Tilt setting how strongly and which way - higher notes louder (or softer), the dynamic key-tracking of real instruments.

T **Vellength** [DYN] [MIDI] in: Audio out: MIDI

Note duration scales with velocity: hard notes ring for Max ms, soft notes for Min (set Max < Min to make accents staccato) - the played articulation where dynamics shape note length.

T **Velocity** [DYN] [MIDI] in: Audio out: MIDI

Scales and offsets note velocity, with a modulation amount.

Expression (3)

☐ **Fall** [FX] [MIDI] in: Audio out: MIDI

Jazz fall-off: on release the pitch sweeps down by Depth over Time while the note rings, then the note releases and the bend snaps back to centre. The note-off is delayed to the end of the fall.

☐ **MidiVibrato** [FX] [MIDI] in: Audio out: MIDI

A pitch-bend LFO that fades in after an Onset delay while any note is held - the delayed, played-in vibrato of a string or voice rather than an instant wobble. Rate in Hz, Depth as bend amount.

☐ **Scoop** [FX] [MIDI] in: Audio out: MIDI

Bends up into each struck note from Depth below, resolving to pitch over Time - the vocal/brass scoop, a pitch-bend ramp that ends at centre so no bend is left hanging.

Harmony (21)

⋮ **AutoInvert** [HRM] [MIDI] in: Audio out: MIDI

Cycles the inversion of each struck chord: every trigger rotates which of the lowest notes jump an octave (by Rotate), so a repeated chord keeps climbing through its inversions - movement ChordVoicing's static shapes don't give.

⋮ **Bassline** [HRM] [MIDI] in: Audio out: MIDI

Follows the chord with a mono bass voice: the lowest held note dropped Octaves down, retriggered whenever the lowest note changes. Passes the original notes through and adds the bass beneath them.

⌵ **Chord**_[HRM]_[MIDI] in: Audio out: MIDI

Turns each incoming note into a chord (165 types) with inversion, spread, octave doubling and strum.

⌵ **ChordLock**_[HRM]_[MIDI] in: Audio out: MIDI

Quantizes every note to the nearest tone of a fixed chord (Root + Type), tighter than Scale's scale-quantize - locks an improvisation to a chord's notes.

⌵ **ChordMem**_[HRM]_[MIDI] in: Audio out: MIDI

Each key plays a stored chord shape (semitone intervals from node config).

⌵ **ChordProg**_[HRM]_[MIDI] in: Audio out: MIDI

Each note triggers the next chord in a stored root progression (node config).

⌵ **ChordVoicing**_[HRM]_[MIDI] in: Audio out: MIDI

Re-voices each note into a jazz shape (open / shell / quartal / spread).

⌵ **Counterpoint**_[HRM]_[MIDI] in: Audio out: MIDI

Adds a second voice in contrary motion - when the melody rises the counter voice falls and vice-versa - quantised to Root/Scale. Interval sets the starting separation.

⌵ **Diatonic**_[HRM]_[MIDI] in: Audio out: MIDI

Transposes by scale DEGREES within Root/Scale (a third, a fifth...) instead of by raw semitones like Transpose - the shift always lands back in key. Mod Amt adds a modulatable degree offset.

⌵ **DiatonicChord**_[HRM]_[MIDI] in: Audio out: MIDI

Auto-harmoniser: builds the in-key chord on every note by stacking diatonic thirds (triad, 7th...) along Root/Scale, so single-finger melodies become correct chords in the key.

⌵ **Drone**_[HRM]_[MIDI] in: Audio out: MIDI

Doubles every note with a fixed pedal root.

⌵ **Glissando**_[HRM]_[MIDI] in: Audio out: MIDI

Fills a stepwise run between consecutive notes.

⌵ **Harmonize**_[HRM]_[MIDI] in: Audio out: MIDI

Adds up to three fixed-interval harmony voices to each note.

⚙️ **Invert**[HRM][MIDI] in: Audio out: MIDI

Mirrors pitch around a pivot note.

⚙️ **MidiCluster**[HRM][MIDI] in: Audio out: MIDI

Adds symmetric tone-cluster voices above and below each note.

⚙️ **MidiFold**[HRM][MIDI] in: Audio out: MIDI

Folds out-of-range notes back inside a Low..High window.

⚙️ **NeoRiemann**[HRM][MIDI] in: Audio out: MIDI

Neo-Riemannian triad transforms: builds a major/minor triad on each note and applies P (parallel), L (leading-tone) or R (relative) - the maximally-smooth chord moves of late-Romantic and film harmony, each sharing two common tones with the input.

⚙️ **Octave**[HRM][MIDI] in: Audio out: MIDI

Doubles each note up and/or down by whole octaves.

⚙️ **Scale**[HRM][MIDI] in: Audio out: MIDI

Quantizes incoming notes to the nearest degree of the chosen root and scale.

⚙️ **Transpose**[HRM][MIDI] in: Audio out: MIDI

Shifts every note by a fixed semitone amount plus a modulatable offset.

⚙️ **VoiceLead**[HRM][MIDI] in: Audio out: MIDI

Smooth voice leading: places each note's pitch class in the octave nearest the previous output, so a line or chord progression moves by the smallest interval. Range caps the leap; past it the original octave is kept.

Modulation (13)

🌊 **CCGlide**[MOD][MIDI] in: Audio out: MIDI

Portamento for a CC: smooths the watched CC toward each newly received value over Time, emitting interpolated CC each block - de-zippers a stepped controller, where MidiSlew only glides the internal scalar.

🌊 **CCLFO**[MOD][MIDI] in: Audio out: MIDI

Continuous CC LFO generator (sine/triangle/saw/square) emitted on a chosen CC at a free Rate - automates a synth parameter over MIDI, where MidiLFO drives only the internal mod-matrix scalar.

🌊 **CCSeq**[MOD][MIDI] in: Audio out: MIDI

Tempo-synced step CC sequencer: one CC value per synced step cycling an 8-step shape (overridable via the node's config list) - a modulation sequencer in the CC domain, where StepSeqMidi sequences notes.

🎵 **Generative** [MOD] [MIDI] in: Audio out: MIDI

Probabilistic melodic random walk synced to a rate.

🎵 **MidiEnv** [MOD] [MIDI] in: Audio out: MIDI

ADSR envelope triggered by the note gate, on the scalar control output.

🎵 **MidiLFO** [MOD] [MIDI] in: Audio out: MIDI

Low-frequency modulation source (sine/tri/saw/square) on the scalar control output.

🎵 **MidiRandom** [MOD] [MIDI] in: Audio out: MIDI

Generates random notes within a pitch range at a synced rate.

🎵 **MidiSampleHold** [MOD] [MIDI] in: Audio out: MIDI

Samples and holds the incoming control value on each note.

🎵 **MidiSlew** [MOD] [MIDI] in: Audio out: MIDI

Smooths (glides) the control value over a set time.

🎵 **PitchDrift** [MOD] [MIDI] in: Audio out: MIDI

Nudges each note's pitch by a small random detune up to +/-Cents (per-note, polyphonic) - the gentle oscillator instability of analog gear, applied to pitch where Humanize only scatters timing and velocity.

🎵 **PressSwell** [MOD] [MIDI] in: Audio out: MIDI

Generates a channel-pressure (aftertouch) swell that rises to Max over Rise while any note is held, snapping back to zero on release - an automatic crescendo for pads and strings, where ATProc only transforms incoming aftertouch.

🎵 **RandOctave** [MOD] [MIDI] in: Audio out: MIDI

Randomly shifts notes by up to +/- Range octaves with a chance.

🎵 **VelToCC** [MOD] [MIDI] in: Audio out: MIDI

Turns each note's velocity into a CC value emitted just before the note, so a synth's filter or level opens with how hard you played - dynamics routed to expression, where Note2CC sends pitch.

Probability (1)

 **Chance** [PRB] [CV] [MIDI] in: Audio out: MIDI

Probabilistic gate: lets each note through with the set probability.

Routing (11)

—< **Channel** [RTE] [MIDI] in: Audio out: MIDI

Reassigns notes to a fixed MIDI channel.

—< **FixedNote** [RTE] [MIDI] in: Audio out: MIDI

Forces every note to a single fixed pitch (drum / trigger).

—< **KeySwitch** [RTE] [MIDI] in: Audio out: MIDI

Splits the keyboard at a note; one zone passes, the other is filtered.

—< **Merge** [RTE] [MIDI] in: Audio out: MIDI

Passes both note-stream inputs through, merged into one stream.

—< **MidiLatch** [RTE] [MIDI] in: Audio out: MIDI

Holds notes after release until the next note arrives.

—< **NoteFilter** [RTE] [MIDI] in: Audio out: MIDI

Passes only notes inside a key and velocity window.

—< **NoteLimit** [RTE] [MIDI] in: Audio out: MIDI

Polyphony cap: never lets more than Max notes sound at once, stealing the oldest (FIFO note-off) when a new note would exceed the limit - tames dense chords and runaway generators.

—< **PolyChan** [RTE] [MIDI] in: Audio out: MIDI

Round-robins each new note to the next of Voices MIDI channels from Base, spreading a polyphonic part across a multitimbral rig - one note per channel, the classic voice-allocation split (distinct from MPEConvert's per-note MPE channels).

—< **Split** [RTE] [MIDI] in: Audio out: MIDI

Keyboard split: notes below the split point transpose by Low, the rest by High.

—< **Thin** [RTE] [MIDI] in: Audio out: MIDI

Note-density limiter: drops any note-on arriving within MinGap ms of the last one passed, decluttering a busy stream or fast repeats - distinct from Notelimit, which caps simultaneous polyphony. A dropped note's off is dropped too.

—< **Transformer** [RTE] [MIDI] in: Audio out: MIDI

Rule: notes within a range are transposed and velocity-scaled.

Sequencing (39)

■□□ **Appoggiatura** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Leaning grace note: each struck note first sounds a long auxiliary (Interval away) that takes Time from the principal, which then sustains - the expressive long grace, unlike the quick Flam or Mordent.

■□□ **Arp** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Arpeggiator: cycles held notes through a pattern at a synced rate with gate, octaves, swing and step count. Latch holds the chord so it keeps arpeggiating after the keys are released (the next key starts a new chord).

■□□ **Cascade** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Each struck note fans out into a timed run of Steps notes climbing (or falling) the scale from it - an arpeggiated flourish fired per key, unlike Arp which cycles the whole held chord.

■□□ **ChordArp** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Cthulhu-style chord arpeggiator: each key builds a full chord (Type) and the result is arpeggiated upward across Octaves at a synced Rate - one finger plays a moving arpeggio of the whole chord. Releasing the key stops its run.

■□□ **Drunk** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Random-walk melodic sequencer: each synced Rate step nudges the pitch by a random +/-Step semitones, bounded within +/-Range of the lowest held note - Brownian melody anchored to the chord, unlike MidiRandom's memoryless picks.

■□□ **Echo** [SEQ] [CV] [MIDI] in: Audio out: MIDI

MIDI delay: repeats notes at a synced time with velocity feedback decay.

■□□ **EuclidMel** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Euclidean melody: spreads Pulses evenly across Steps (Bjorklund) and, on each hit, advances a cursor through the held chord - the rhythm is Euclidean and the pitch walks the chord, unlike SeqEuclid which stabs all held notes per pulse.

■□□ **Flam** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Precedes every note with a quiet grace note, delaying the main hit so the grace leads it - the classic drum flam.

■□□ **GateArp** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Cthulhu's arp gate lane: walks the held chord upward across Octaves while the note length cycles a per-step staccato/legato pattern, so the rhythm breathes - short stabs then sustained notes. Gate scales the pattern (>1 overlaps into the next step).

■□□ **MidiBrianBrain** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Brian's Brain MIDI sequencer: an 8x8 three-state grid (ready / firing / dying) evolves under Brian's Brain rules - a ready cell ignites only with exactly two firing neighbours, firing cells pass to dying, dying cells reset. Each clock step plays a note for every firing cell in the next column (row maps to a scale degree from Root/Scale); a full 8-step sweep advances one generation. The refractory state keeps it restless and sparkly where Game of Life settles. Density sets the random fill. The MIDI-note counterpart to the BrianBrain CV block.

■□□ **MidiGameOfLife** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Conway's Game of Life MIDI sequencer: an 8x8 cell grid evolves under the B3/S23 rule; each clock step reads the next column and plays a note for every live cell (row maps to a scale degree from Root/Scale), and a full 8-step sweep advances one generation. Gliders, blinkers and still-lives become ever-shifting melodic and harmonic patterns. Density sets the random fill when the grid (re)seeds. The MIDI-note counterpart to the GameOfLife CV block.

■□□ **MidiLangton** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Langton's Ant MIDI melody: a single turmite walks an 8x8 grid - turning right on a white cell, left on a black one, flipping each cell as it leaves, then stepping forward. Each clock advances the ant Steps cells and plays one note for its current row (mapped to a scale degree from Root/Scale), so the melody scribbles chaotically then locks into the famous periodic 'highway'. Emergent order from one trivial rule - a single moving agent, not a population.

■□□ **MidiMarkov** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Learns the note-to-note transition statistics of what you play (a 12x12 pitch-class chain) and, on each synced step, walks the chain to improvise a new line in the same style. Memory fades old transitions.

■□□ **MidiQuantize** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Snaps note starts to a synced grid by a strength amount.

■□□ **MidiTranceGate** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Chops the held notes with a 16-step on/off Pattern at a synced Rate: every 'on' step retriggers all held notes, gated to a fraction of the step. The rhythmic gate behind the trance sound - pattern-driven, where Roll is a fixed pulse.

■□□ **MidiTuring** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Turing-machine sequencer (Music Thing style): a circular shift register clocks once per step and the held notes fire when the read bit is 1. Change is the probability the recycled bit flips - 0 locks an evolving loop, 1 fully randomises, between = slowly mutating melodies.

■□□ **Mordent** [SEQ] [CV] [MIDI] in: Audio out: MIDI

One-shot ornament: each struck note plays principal -> auxiliary -> principal at the attack, then sustains, the single-flick counterpart to the continuous Trill.

■□■ **NoteLength** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Forces a fixed note duration (gate length).

■□■ **NoteOffDelay** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Extends each note's gate by delaying its note-off.

■□■ **OctaveArp** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Cthulhu's arp octave lane: arpeggiates the held chord while a per-step octave-offset Pattern leaps the line up and down by whole octaves (alternating, staircase, bounce...) instead of climbing monotonically like StepArp.

■□■ **PatternGate** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Gates note-ons through a 16-step on/off pattern (bitmask).

■□■ **Polymeter** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Two step lanes of different lengths (LenA, LenB) clock together over the held notes, phasing against each other so the combined pattern only repeats after lcm(LenA, LenB) steps - instant polymetric movement from one chord.

■□■ **RatchetArp** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Cthulhu's arp repeat lane: walks the held chord upward across Octaves while a per-step pattern retriggers selected steps multiple times (capped by Max), packing rolls into the line. Unlike SeqRatchet's fixed count, the repeat count varies per step.

■□■ **Ricochet** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Accelerating bouncing retriggers with decaying velocity.

■□■ **Roll** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Drum roll / buzz: retriggers every held note at Rate for as long as it is held, each hit gated to a fraction of the interval.

■□■ **SeqEuclid** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Euclidean rhythm gate: spreads Pulses evenly across Steps (rotatable) at a synced rate.

■□■ **SeqRatchet** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Retriggers each note Count times within a synced step.

■□■ **Shift** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Delays every note by Delay ms (groove nudge / lay-back), pushing note-ons and offs back by the same amount so durations are preserved - the per-track timing offset of a groove sequencer.

■□□ **SkipArp** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Cthulhu's arp skip lane: arpeggiates the held chord upward across Octaves, but each step has a Skip probability of being dropped to a rest, thinning the pattern differently every cycle for generative movement.

■□□ **Spiral** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Transposing MIDI delay: each repeat is delayed by Time and shifted by Interval semitones with velocity decaying by Feedback, so one note spins off a climbing or falling staircase of echoes - unlike Echo, which repeats the same pitch.

■□□ **StepArp** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Cthulhu-style step arpeggiator: walks the held chord strictly upward across Octaves octaves at a synced Rate, accenting the first step of each cycle - the rolling, octave-jumping arp of Xfer Cthulhu's sequencer. Distinct from Arp's mode-cycling: this climbs the chord in order.

■□□ **StepSeqMidi** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Melodic step sequencer - semitone offsets (node config) clock the held root.

■□□ **Strum** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Spreads a chord's notes over time, up or down, like a guitar strum.

■□□ **Swing** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Pushes off-beat eighth-notes later for a swing/shuffle feel.

■□□ **TransArp** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Cthulhu's arp transpose lane: arpeggiates the held chord upward across Octaves while a per-step semitone Pattern (fifth pedals, triad arps, scalar runs) is added on top, so the line outlines a melodic contour, not just the chord tones.

■□□ **Trill** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Keyboard trill: while a note is held, alternates rapidly between it and the note Interval semitones above, at Rate.

■□□ **Tuplet** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Snaps note starts to a Tuples-per-beat grid (3 = triplets, 5 = quintuplets) by Strength, for a triplet or odd-tuplet feel where MidiQuantize only snaps to the straight grid.

■□□ **Turn** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Gruppetto turn: a four-step ornament at the attack - upper auxiliary, principal, lower auxiliary, then the principal held. Time sets each step, Interval the neighbour distance.

▣▣▣ **VelArp** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Cthulhu's arp velocity lane as a ramp: arpeggiates the held chord while velocity sweeps from From to To across Steps then resets, so every pass swells or fades without a per-step editor.

Tuning (3)

└ **JustTune** [TUN] [MIDI] in: Audio out: MIDI

Adaptive just intonation: retunes each note toward its 5-limit JI ratio relative to Key via per-note detune (polyphonic), Amount blending from equal temperament to pure - the beatless thirds and fifths of real harmony.

└ **Microtuner** [TUN] [MIDI] in: Audio out: MIDI

Stretched-octave microtuning via per-note pitch detune.

└ **TuneTable** [TUN] [MIDI] in: Audio out: MIDI

Custom microtuning: detunes each note by a per-pitch-class cents offset from the node's 12-value config list (relative to Key) - meantone, Pythagorean, maqam or gamelan, any temperament. Amount scales the table.

Voicing (5)

⊗ **MidiUnison** [VOI] [MIDI] in: Audio out: MIDI

Doubles each note into multiple voices, optionally spread across channels.

⊗ **Mono** [VOI] [MIDI] in: Audio out: MIDI

Monophonic note priority (last / lowest / highest).

⊗ **MPEConvert** [VOI] [MIDI] in: Audio out: MIDI

Assigns each note its own rotating channel for per-note MPE expression.

⊗ **Slide** [VOI] [MIDI] in: Audio out: MIDI

Cthulhu's slide: mono legato glue. Picks one held note by Priority (last/low/high) and emits the new note-on before releasing the old one, so a portamento synth slurs between notes instead of retriggering.

⊗ **Sustain** [VOI] [MIDI] in: Audio out: MIDI

Sustain-pedal simulation: while Hold is engaged, note-offs are captured so the notes keep sounding; when Hold drops, every held note releases at once. Hold is a parameter, so a CC, an LFO or automation can play the pedal.

Index (111 blocks, alphabetical)

